RippleLab Day 05 Progress Report

Day: 05 of 30

Date: 2026-08-13

Status: Complete

Branch: `codex/day-05-contracts`

Implementation commit: `4745f97`

Quality gate: Full local suite passed

1. Objective

Define one versioned contract that the RippleLab web application and economic engine can share for scenario requests, deterministic results, causal graphs, assumptions, evidence and confidence. Unsupported scenario types and incorrect units must fail before any calculation runs.

2. Acceptance Criteria

- [x] Versioned request and result schemas are canonical and closed to unknown fields.
- [x] All five MVP scenario families have stable enum values and exact shock units.
- [x] Causal nodes and edges carry mechanism, lag, evidence, assumption and confidence references.
- [x] Citation metadata and a scored confidence rubric are defined.
- [x] TypeScript types are generated from the canonical JSON Schema.
- [x] CI detects stale generated TypeScript.
- [x] The FastAPI boundary accepts the same golden request used by the contract validator.
- [x] Unsupported scenarios, wrong units and extra fields fail clearly in JavaScript and Python tests.
- [x] A complete repo-rate golden request/result is documented.

3. Completed Work

RippleLab now has a strict `1.0.0` simulation boundary. The request contains an immutable profile snapshot, a discriminated scenario, explicit assumptions and evidence. The result contains formula/model versioning, individual impacts, annual net impact in paise, uncertainty bounds, a causal graph, citations, warnings and a confidence score with rationale.

The five supported scenarios are inflation change, RBI repo-rate change, oil-price change, income-tax change and job-income change. Each uses a fixed semantic unit. A repo-rate shock must use basis points; callers cannot label it in percentage points and rely on an implicit conversion.

A public `/progress` page was also added so the deployed site can show the actual delivery state without requiring authentication or exposing financial data.

4. Contract Architecture

Layer	Artifact	Responsibility
Canonical	JSON Schema Draft 2020-12	Defines accepted shapes, enums, units and limits
Web	Generated TypeScript	Provides compile-time request/result types and constants
API	Pydantic models	Enforces the same boundary at runtime
Examples	Versioned JSON request/result	Provides reproducible golden fixtures
Tests	JavaScript and pytest	Proves unit/type rejection and cross-language agreement

The generated TypeScript file is committed for consumers but contains a generated-file warning. `pnpm generate:contracts` updates it; the package check fails if the checked-in output no longer matches the schema.

5. Supported Scenario Units

Scenario	Contract enum	Required shock unit
Inflation	`inflation_change`	Percentage points
RBI repo rate	`repo_rate_change`	Basis points
Oil price	`oil_price_change`	USD per barrel
Effective income tax	`income_tax_change`	Percentage points
Job loss or salary	`job_income_change`	Percent of income

All money fields ending in `Paise` are integers. Rates ending in `Bps` are integer basis points. The request fixes country to India and currency to INR for the first MVP.

6. Confidence Rubric

Five dimensions are independently scored 0-4: evidence quality, causal directness, model stability, personalization coverage and data recency. The total score is the dimension sum multiplied by five.

- 0-39: Low
- 40-79: Medium
- 80-100: High

The validator checks that each stored score equals its dimensions and that the label matches the score band. Every confidence object also requires a plain-language rationale. This score describes support for an estimate under stated assumptions; it is not a probability that a forecast will occur.

7. Golden Repo-Rate Example

The golden request models a 100 basis-point repo-rate cut for a salaried Bengaluru homeowner with a floating home loan and fixed deposits. It snapshots the personal balances, declares loan/deposit pass-through ratios and cites RBI source metadata.

The illustrative result shows two opposing causal paths: lower floating-loan payments and lower fixed-deposit income. Its INR 18,000 annual net benefit and uncertainty bounds are contract fixtures, not Day 6 validated

calculations. The explicit warning prevents the example from being mistaken for implemented economic logic.

Every edge references existing source/target nodes, assumptions and citations. Every impact references a causal node. The validator checks those relationships in addition to ordinary JSON shape validation.

8. API Agreement

FastAPI exposes `POST /v1/contracts/simulation/validate`. It parses the golden request with a discriminated Pydantic union and returns only its validated schema version, scenario type and request ID. The endpoint intentionally does not calculate an outcome.

Python tests confirm acceptance of the same golden request and rejection of:

- unsupported `gst_change` scenario input;
- `percentage_points` supplied where repo-rate `basis_points` are required;
- undocumented/extra scenario fields.

9. Verification Evidence

Check	Method	Result
Canonical schema	Draft, required definitions and closed-object assertions	Passed
Golden request/result	Structural and relational validation	Passed
Failure cases	Scenario, unit and version rejection	Passed
Generated TypeScript	Deterministic stale-file comparison	Passed
FastAPI agreement	Same golden request through Pydantic endpoint	Passed
API tests	Ruff plus pytest	6 passed
Web quality	ESLint, TypeScript and Next.js production build	Passed
Browser suite	Desktop and mobile Chromium	19 passed; 3 intentional device skips
Accessibility	axe-core including public progress page	No serious or critical violations
Profile security	Migration/RLS policy assertion	Passed
Secret scan	Repository scan	Passed
Full quality gate	`pnpm check`	Passed
Visual review	Public progress page in in-app browser	Passed

10. Important Files

- `packages/contracts/schemas/simulation-contract.schema.json` - canonical contract.
- `packages/contracts/src/generated/simulation.ts` - generated web types and constants.
- `packages/contracts/examples` - repo-rate golden request and result.
- `packages/contracts/scripts` - type generator and structural/relational validator.
- `services/economic-engine/src/ripplelab_engine/contracts.py` - strict API mirror.

- `services/economic-engine/tests/test_simulation_contract.py` - API agreement and rejection tests.
- `docs/SIMULATION_CONTRACT_V1.md` - versioning, units, confidence and example documentation.
- `apps/web/src/app/progress/page.tsx` - public progress preview.

11. Visual Review

The public progress page presents five completed foundations, Day 6 as the next checkpoint and Day 7 as the first end-to-end simulation target. It includes a compact causal-contract illustration showing that a policy value is followed by an assumption/source link and a personal impact/confidence link. The page is responsive and passes desktop and mobile automated accessibility checks.

12. Decisions

- Use JSON Schema Draft 2020-12 as the canonical cross-language artifact.
- Use discriminated scenario objects rather than a single free-form shock shape.
- Require exact units so conversions are deliberate and reviewable.
- Snapshot relevant profile values in a request for reproducible saved results.
- Exclude profile goals and direct identity fields because calculations do not need them.
- Separate transport schema version from model/formula version.
- Treat confidence as a transparent evidence rubric, not a forecast probability.
- Keep the Day 5 API endpoint validation-only; economic calculations begin Day 6.

13. Risks and Blockers

The Python model is a maintained runtime mirror, while TypeScript is generated directly. Contract agreement tests reduce drift, and Day 29 can replace the Python mirror with automated model generation if the toolchain remains stable.

The golden result uses illustrative monetary outcomes until Day 6 calculation primitives are complete. It is labeled accordingly in both documentation and result warnings.

Hosted authentication/profile persistence still requires a linked Supabase project. The public progress and design-system routes remain fully usable without it.

No code blocker remains for Day 5.

14. Next Day

Day 6 will implement Decimal-safe EMI, outstanding-balance, floating-rate reset, deposit-interest and rate pass-through primitives. Every function will publish its formula version, rounding rule and assumptions and will be tested against known examples and boundaries.

Sign-off

Prepared by: Codex

Evidence reviewed: Yes

Ready to continue: Yes